

```
face2 = AsciiArt.fromFile('face.txt')
face2.display()
```

The `AsciiArt` class has an `__init__()` method that can be passed the text characters of the image as a string. It also has a `fromFile()` class method that can be passed the filename string of a text file containing the ASCII art. Both methods create `AsciiArt` objects.

When you run this program and there is a *face.txt* file that contains the ASCII art face, the output will look something like this:

```
  _____
 |   . .   |
 |  \  /   |
 |_____|   |
 
  _____
 |   . .   |
 |  \  /   |
 |_____|   |
```

The `fromFile()` class method makes your code a bit easier to read, compared to having `__init__()` do everything.

Another benefit of class methods is that a subclass of `AsciiArt` can inherit its `fromFile()` method (and override it if necessary). This is why we call `cls(characters)` in the `AsciiArt` class's `fromFile()` method instead of `AsciiArt(characters)`. The `cls()` call will also work in subclasses of `AsciiArt` without modification because the `AsciiArt` class isn't hardcoded into the method. But an `AsciiArt()` call would always call `AsciiArt` class's `__init__()` instead of the subclass's `__init__()`. You can think of `cls` as meaning “an object representing this class.”

Keep in mind that just as regular methods should always use their `self` parameter somewhere in their code, a class method should always use its `cls` parameter. If your class method's code *never* uses the `cls` parameter, it's a sign that your class method should probably just be a function.

Class Attributes